

RAPPORT DE PROJET

Générateur de Mots de Passe Sécurisés

Application web — HTML · CSS · JavaScript



BINTOUL Giovanni

BTS SIO SLAM — 2ème année

Lycée Melkior-Garré

Année scolaire 2024 – 2026

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 1
Nom, prénom : BINTOUL Giovanni		N° candidat : 2318021672
Épreuve ponctuelle ☒	Contrôle en cours de formation ☐	Date : 08 / 05 / 2026
Organisation support de la réalisation professionnelle Lycée Melkior Garré — BTS SIO 2 option SLAM		
Intitulé de la réalisation professionnelle Générateur de mots de passe sécurisé		
Période de réalisation : Novembre 2025 – Janvier 2026 Lieu : Lycée Melkior-Garré / Domicile		
Modalité : ☒ Seul(e) ☐ En équipe		
Compétences travaillées ☐ Concevoir et développer une solution applicative ☐ Assurer la maintenance corrective ou évolutive d'une solution applicative ☐ Gérer les données		
Conditions de réalisation¹ (ressources fournies, résultats attendus) Ressources fournies : MacBook (macOS), connexion Internet, éditeur VS Code, navigateur Google Chrome. Résultats attendus : Une application web permettant de générer des mots de passe sécurisés personnalisables directement depuis le navigateur, sans installation. Application entièrement réalisée en HTML, CSS et JavaScript. Le projet a été réalisé seul, dans le cadre des travaux pratiques de BTS SIO 2 option SLAM. Ressources fournies : MacBook (macOS), navigateur Google Chrome, connexion Internet, éditeur VS Code. Résultat attendu : une application web fonctionnelle permettant de générer des mots de passe sécurisés personnalisables directement depuis le navigateur, sans aucune installation requise. Logiciels : Visual Studio Code, Google Chrome DevTools, GitHub (hébergement et déploiement GitHub Pages). Langages : HTML5, CSS3, JavaScript (ES6+). Documentation : MDN Web Docs (Mozilla) pour HTML5, CSS3, JavaScript ; cours BTS SIO SLAM. Matériel : MacBook (macOS), navigateur Google Chrome pour les tests et l'inspection du code. Ressources complémentaires : Stack Overflow, MDN Web Docs, tutoriels YouTube développement web.		

¹ En référence aux conditions de réalisation et ressources nécessaires du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

² Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

Modalités d'accès aux productions³ et à leur documentation⁴

L'application est accessible via GitHub Pages : <https://gio2ni.github.io/g-n-rateur-mdp/>

Le code source complet est disponible sur le dépôt GitHub public (fichiers : index.html, style.css, script.js).

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS**SESSION 2026****Épreuve E5 - Conception et développement d'applications (option SLAM)****ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)**

³ Conformément au référentiel du BTS SIO « Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve. ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

⁴ Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

CONTEXTE ET OBJECTIF

Le projet consiste à développer une application web permettant de générer des mots de passe sécurisés et personnalisables. L'objectif est de fournir un outil simple, accessible depuis n'importe quel navigateur sans installation, qui aide les utilisateurs à créer des mots de passe forts plutôt que de choisir des mots de passe trop simples comme "123456" ou leur prénom.

FONCTIONNALITÉS DÉVELOPPÉES

1. Génération aléatoire d'un mot de passe au chargement et à chaque clic sur le bouton.
2. Choix de la longueur via un curseur (de 4 à 64 caractères) avec mise à jour en temps réel.
3. Sélection des types de caractères : majuscules (A–Z), minuscules (a–z), chiffres (0–9), symboles.
4. Indicateur de sécurité sur 4 niveaux : Faible, Moyen, Fort, Très fort — avec barres colorées.
5. Bouton « Copier » avec message de confirmation « Copié ! » pendant 1,5 secondes.
6. Bouton afficher / masquer le mot de passe (masqué par défaut).
7. Historique des 10 derniers mots de passe générés avec possibilité de copie individuelle.
8. Interface responsive fonctionnant sur ordinateur et téléphone.

TECHNOLOGIES UTILISÉES

- Langages : HTML5 (structure), CSS3 / Flexbox (mise en page et animations), JavaScript ES6+ (logique).
- API navigateur : `navigator.clipboard.writeText()` pour la copie, `window.crypto.getRandomValues()` pour une génération cryptographiquement sûre.
- Outils : Visual Studio Code, Google Chrome DevTools, GitHub / GitHub Pages (déploiement).

RÉSULTATS

L'application est complète, fonctionnelle et déployée en ligne via GitHub Pages. Elle est accessible depuis n'importe quel navigateur moderne sur ordinateur ou téléphone. Toutes les fonctionnalités du cahier des charges sont implémentées, y compris les bonus (historique, afficher/masquer). Aucune donnée n'est envoyée sur Internet : tout se passe localement dans le navigateur.

L'application est complète, fonctionnelle et déployée sur GitHub Pages. Elle est accessible depuis n'importe quel navigateur moderne sur ordinateur ou téléphone. Le projet a permis de mettre en pratique les compétences de développement front-end acquises en BTS SIO SLAM.

1. Introduction	1
2. Présentation du besoin	2
2.1 Contexte général	
2.2 Problèmes identifiés	
2.3 Public cible	
3. Cahier des charges	4
3.1 Fonctionnalités principales	
3.2 Fonctionnalités bonus	
3.3 Contraintes techniques	
4. Choix techniques	6
4.1 Pourquoi HTML / CSS / JavaScript	
4.2 Environnement de développement	
4.3 Comparatif des solutions	
5. Architecture du projet	8
5.1 Structure des fichiers	
5.2 Organisation du code JavaScript	
6. Développement	9
6.1 La génération du mot de passe	
6.2 Le calcul de la force	
6.3 Les options de personnalisation	
6.4 Le bouton Copier	
6.5 L'historique des mots de passe	
6.6 L'interface et les animations	
7. Sécurité et bonnes pratiques	12
8. Tests et validation	14
8.1 Tests manuels réalisés	
8.2 Compatibilité navigateurs	
9. Difficultés rencontrées	16
10. Solutions apportées	18
11. Résultat final	19
12. Conclusion et perspectives	20

1. Introduction

Dans le cadre de ma deuxième année de BTS SIO SLAM au lycée Melkior-Garré, j'ai réalisé un projet de développement web : un générateur de mots de passe sécurisé. Ce projet m'a permis de mettre en pratique les connaissances acquises tout au long de ma formation, notamment en HTML, CSS et JavaScript.

L'idée est venue d'une observation simple du quotidien : beaucoup de personnes utilisent des mots de passe bien trop faciles à deviner, comme leur prénom, leur date de naissance ou le classique "123456". Pourtant, un mot de passe faible est l'une des premières causes de piratage de comptes en ligne.

Ce rapport présente l'ensemble de la démarche que j'ai suivie : du besoin initial jusqu'au résultat final, en passant par les choix techniques, le développement concret et les difficultés que j'ai rencontrées en chemin.

Informations sur le projet

Type : Application web front-end (côté client uniquement) Technologies : HTML5, CSS3, JavaScript (ES6+)

Durée de développement : environ 3 semaines - Aucune installation requise - Fonctionne directement dans le navigateur

2. Présentation du besoin

2.1 Contexte général

Avec la multiplication des services en ligne — réseaux sociaux, banques, messageries, e-commerce chaque utilisateur doit aujourd'hui gérer des dizaines de comptes. Chacun de ces comptes nécessite un mot de passe, idéalement unique et fort.

Selon plusieurs études en cybersécurité, les mots de passe les plus utilisés dans le monde restent tristement prévisibles : "password", "123456", "azerty". Ces mots de passe peuvent être devinés en quelques secondes par une attaque de type brute force ou par dictionnaire.

Quelques chiffres clés

- 80% des violations de données sont liées à des mots de passe faibles ou réutilisés (Verizon, 2023)
- Le mot de passe "123456" est encore utilisé par plus de 23 millions de personnes dans le monde
- Un mot de passe de 8 caractères peut être cracké en moins de 2 heures avec du matériel moderne
- Un mot de passe de 16 caractères mixtes prendrait plusieurs milliers d'années à cracker

2.2 Problèmes identifiés

En analysant le comportement courant des utilisateurs, j'ai identifié plusieurs problèmes concrets :

- Mots de passe trop simples : les utilisateurs choisissent ce qu'ils retiennent facilement, pas ce qui est sécurisé.
- Réutilisation : un même mot de passe utilisé sur plusieurs sites multiplie les risques.
- Pas de retour visuel sur la sécurité : l'utilisateur ne sait souvent pas si son mot de passe est bon ou non.
- Difficulté à mémoriser les règles : majuscule obligatoire, minimum 8 caractères, symboles... c'est complexe à gérer manuellement.
- Copie fastidieuse : noter un mot de passe généré à la main est source d'erreurs.

2.3 Public cible

Cet outil s'adresse à un public large : n'importe quel internaute souhaitant créer un mot de passe fort rapidement, sans avoir de connaissances techniques particulières. L'interface se veut simple et intuitive pour être accessible à tous.

Problème identifié	Solution apportée
Mots de passe trop simples	Génération aléatoire contrôlée
Pas de retour sur la sécurité	Indicateur de force en temps réel
Copie manuelle fastidieuse	Bouton Copier avec confirmation
Oubli des mots de passe générés	Historique des 10 derniers
Affichage permanent du mdp	Bouton afficher / masquer



3. Cahier des charges

3.1 Fonctionnalités principales

Ces fonctionnalités constituent le cœur de l'application. Elles doivent toutes être présentes dans la version finale.

- Générer un mot de passe aléatoire en cliquant sur un bouton
- Choisir la longueur du mot de passe via un curseur (de 4 à 64 caractères)
- Sélectionner les types de caractères à inclure : majuscules (A-Z), minuscules (a-z), chiffres (0-9), symboles (!@#\$...)
- Afficher le mot de passe généré de manière claire et lisible
- Copier le mot de passe en un clic avec un retour visuel (message « Copié ! »)
- Afficher un indicateur de force du mot de passe sur 4 niveaux : Faible / Moyen / Fort / Très fort
- Mettre à jour le mot de passe automatiquement quand les options changent

3.2 Fonctionnalités bonus

Ces fonctionnalités ont été ajoutées pour améliorer le confort d'utilisation.

- Bouton pour afficher ou masquer le mot de passe (par défaut masqué)
- Historique des 10 derniers mots de passe générés
- Possibilité de copier un mot de passe depuis l'historique
- Bouton pour effacer l'historique entièrement
- Animation lors de la génération (effet de transition fluide)

3.3 Contraintes techniques

- Utiliser uniquement HTML, CSS et JavaScript — aucun framework externe
- Code lisible, structuré et commenté pour être compréhensible en BTS
- Interface responsive : l'application doit fonctionner sur ordinateur et téléphone
- Compatibilité avec les navigateurs modernes (Chrome, Firefox, Edge, Safari)
- Temps de chargement quasi-instantané
- pas de dépendance lourde
- Aucun stockage côté serveur
- toutes les données restent dans le navigateur

Astuce pour l'oral

Présentez d'abord les fonctionnalités principales en faisant une démonstration en direct. Ensuite montrez les bonus. Ça donne une bonne structure : essentiel → confort.





4.1 Pourquoi HTML / CSS / JavaScript

J'ai choisi de travailler avec les trois technologies de base du web, sans framework comme React, Vue ou Angular. Ce choix est délibéré : en BTS SIO SLAM, on apprend ces fondamentaux en première, et je voulais vraiment les maîtriser avant de passer à des outils plus abstraits.

De plus, pour un projet de cette taille, utiliser un framework aurait été surdimensionné.

Vanilla JavaScript est parfaitement adapté, plus rapide à charger et plus simple à expliquer à l'oral.

Technologie	Rôle dans le projet
HTML5	Structure sémantique : boutons, inputs, listes, zones d'affichage
CSS3	Mise en page Flexbox/Grid, design, couleurs, animations, responsive
JavaScript	Logique : génération, copie, historique, DOM, indicateur de force

4.2 Environnement de développement

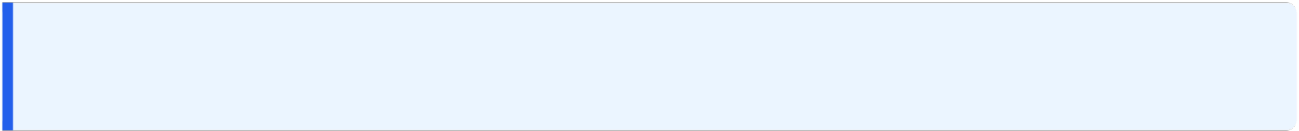
Pour développer le projet, j'ai utilisé Visual Studio Code (VS Code), l'éditeur de code le plus populaire du marché. Il est gratuit, léger et très complet avec ses extensions. J'ai notamment utilisé l'extension Live Server pour voir mes modifications en temps réel dans le navigateur sans avoir à rafraîchir la page.

Outil	Utilisation
Visual Studio Code	Éditeur de code principal
Extension Live Server	Prévisualisation en temps réel
Google Chrome DevTools	Débogage JavaScript et inspection CSS
GitHub (optionnel)	Versionnage du code source

4.3 Comparatif des solutions envisagées

Avant de commencer, j'ai réfléchi à plusieurs approches possibles pour réaliser ce projet :

Solution	Avantages	Inconvénients
HTML/CSS/JS pur (choix retenu)	Simple, rapide, pas de dépendances, idéal pour le BTS	Moins de confort que les frameworks pour de gros projets
React.js	Composants réutilisables, très utilisé en entreprise	Trop complexe pour ce projet, nécessite Node.js
Application Python (Flask)	Back-end possible, plus puissant	Nécessite un serveur, plus lourd à déployer



5. Architecture du projet

5.1 Structure des fichiers

Le projet est organisé de manière simple et claire, avec trois fichiers distincts selon le principe de séparation des responsabilités :

generateur-mdp/ ■ ■ ■ index.html → Structure de la page (HTML) ■ ■ ■ style.css → Mise en forme et design (CSS) ■ ■ ■ script.js → Logique et interactions (JavaScript)

Cette séparation rend le code plus facile à lire, à modifier et à déboguer. On sait exactement quel fichier ouvrir selon le type de modification à faire.

5.2 Organisation du code JavaScript

Le fichier script.js est organisé en sections clairement délimitées par des commentaires. Chaque section a un rôle précis :

Section	Rôle
1. Sélection du DOM	Récupération de tous les éléments HTML nécessaires
2. Jeux de caractères	Définition des alphabets (majuscules, chiffres, symboles...)
3. Génération	Fonction principale de construction du mot de passe
4. Vérification équilibre	Garantit la présence de chaque type de caractère
5. Calcul de la force	Évaluation du mot de passe sur plusieurs critères
6. Mise à jour de l'interface	Affichage du résultat et de l'indicateur de force
7. Gestionnaires d'événements	Réactions aux clics, changements de cases, slider
8. Historique	Ajout, affichage et suppression des entrées

Section	Rôle
9. Initialisation	Génération automatique au chargement de la page

Bonne pratique

Séparer le code en sections commentées est une habitude professionnelle essentielle. Ça permet à n'importe quel développeur (ou correcteur) de comprendre rapidement le code.

6. Développement

6.1 La génération du mot de passe

C'est la partie la plus importante du projet. La logique repose sur un principe simple mais efficace : construire un « pool » de caractères disponibles à partir des options choisies, puis en sélectionner aléatoirement autant que la longueur voulue.

Étape 1 — Construction du pool :

On commence avec une chaîne vide, et on y ajoute les jeux de caractères correspondant aux cases cochées :

```
let pool = '';
if (optUppercase.checked) pool += 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
if (optLowercase.checked) pool += 'abcdefghijklmnopqrstuvwxyz';
if (optNumbers.checked) pool += '0123456789';
if (optSymbols.checked) pool += '!@#$%^&*()_+=[ ]{}|;:,.<>?';
```

Étape 2 — Tirage aléatoire :

On répète l'opération suivante autant de fois que la longueur souhaitée :

```
for (let i = 0; i < length; i++) {
  const index = Math.floor(Math.random() * pool.length);
  password += pool[index];
}
```

Explication de Math.random()

Math.random() retourne un nombre décimal entre 0 (inclus) et 1 (exclus). En le multipliant par pool.length, on obtient un nombre entre 0 et la longueur du pool. Math.floor() arrondit à l'entier inférieur pour obtenir un index valide.

6.2 Le calcul de la force du mot de passe

La force d'un mot de passe est évaluée en attribuant des points selon plusieurs critères. Plus le score est élevé, plus le mot de passe est considéré comme sécurisé.

Critère	Condition	Points
Longueur minimale	≥ 8 caractères	+1
Longueur correcte	≥ 12 caractères	+1

Critère	Condition	Points
Longueur excellente	≥ 20 caractères	+1
Contient une majuscule	Regex [A-Z]	+1
Contient une minuscule	Regex [a-z]	+1
Contient un chiffre	Regex [0-9]	+1
Contient un symbole	Regex [^A-Za-z0-9]	+1

Score total	Niveau affiché
0 à 2 points	Faible
3 à 4 points	Moyen
5 à 6 points	Fort
7 points	Très fort

Les expressions régulières (Regex) permettent de tester rapidement si un mot de passe contient un type de caractère donné. Par exemple, `/[A-Z]/.test(password)` retourne `true` si le mot de passe contient au moins une lettre majuscule.

6.3 Les options de personnalisation

Les quatre options (majuscules, minuscules, chiffres, symboles) sont des cases à cocher HTML (input type checkbox). En JavaScript, on lit leur état avec la propriété `.checked`. Dès qu'une case est modifiée, un événement 'change' est déclenché et un nouveau mot de passe est automatiquement généré.

```
[optUppercase, optLowercase, optNumbers, optSymbols].forEach(opt => {
  opt.addEventListener('change', () => {
    const newPassword = generatePassword();
    updateUI(newPassword);
  });
});
```

6.4 Le bouton Copier

La copie dans le presse-papiers utilise l'API native du navigateur : `navigator.clipboard.writeText()`. Cette fonction est asynchrone, ce qui veut dire qu'elle ne bloque pas le reste du code pendant son exécution. Elle retourne une Promise.

```
navigator.clipboard.writeText(text)
  .then(() => {
    copyFeedback.classList.add('show');
    setTimeout(() => {
      copyFeedback.classList.remove('show');
    }, 1500);
  })
  .catch(err => console.error('Erreur copie :', err));
```


Point important

L'API Clipboard nécessite un contexte sécurisé pour fonctionner. En local (fichier ouvert depuis le bureau) : peut ne pas fonctionner selon le navigateur. En production (serveur HTTPS ou localhost) : fonctionne parfaitement.

6.5 L'historique des mots de passe

À chaque génération, le nouveau mot de passe est ajouté au début d'un tableau JavaScript. Si le tableau dépasse 10 entrées, la plus ancienne est supprimée automatiquement. La liste HTML est ensuite entièrement reconstruite depuis ce tableau.

```
// Ajout en début de tableau
history.unshift(newPassword);
// Limite à 10 entrées
if (history.length > 10) history.pop();
// Reconstruction de la liste HTML
historyList.innerHTML = '';
history.forEach(pwd => {
  const li = document.createElement('li');
  li.textContent = pwd;
  historyList.appendChild(li);
});
```

6.6 L'interface et les animations

Le design est pensé pour être épuré et agréable à utiliser. Les couleurs sont douces, le layout est centré, et les éléments sont bien espacés. Les animations sont entièrement réalisées en CSS (transitions, @keyframes) pour ne pas alourdir le JavaScript.

- Transition sur les boutons : effet de survol (hover) avec légère élévation
- Animation du mot de passe : effet de scan au moment de la génération
- Barre de force : la largeur et la couleur changent progressivement avec transition —
- Message Copié ! : apparition/disparition avec opacité animée
- Options cochées : la carte change de couleur de fond quand l'option est active

7. Sécurité et bonnes pratiques

Même si ce projet est une application front-end simple, il est important de respecter certaines bonnes pratiques liées à la sécurité informatique.

Randomisation

La fonction `Math.random()` de JavaScript est suffisante pour générer des mots de passe à usage personnel. Elle n'est cependant pas "cryptographiquement sûre", ce qui veut dire qu'elle n'est pas recommandée pour des applications nécessitant une sécurité maximale. Pour aller plus loin, on pourrait utiliser `window.crypto.getRandomValues()`, qui est cryptographiquement sûr.

Méthode	Usage recommandé
Math.random()	Mots de passe personnels, usage courant
window.crypto.getRandomValues()	Applications nécessitant une sécurité maximale

Confidentialité des données

L'un des points forts de cette application, c'est que les mots de passe générés ne quittent jamais le navigateur de l'utilisateur. Il n'y a pas de serveur, pas de base de données, pas d'envoi réseau. Tout se passe localement.

Principe Privacy by Design

Aucun mot de passe n'est envoyé sur Internet. Aucune donnée personnelle n'est collectée ou stockée. L'historique disparaît dès que l'utilisateur ferme l'onglet. Ce principe s'appelle le Privacy by Design : la vie privée est protégée dès la conception.

Conseils à l'utilisateur

- Utiliser un mot de passe différent pour chaque site ou service
- Préférer des mots de passe d'au moins 12 caractères avec les 4 types de caractères
- Stocker les mots de passe dans un gestionnaire dédié (Bitwarden, KeePass...)
- Ne jamais noter un mot de passe sur un papier ou dans un fichier texte non chiffré
- Activer l'authentification à deux facteurs (2FA) quand c'est possible

8. Tests et validation

8.1 Tests manuels réalisés

Pour valider le bon fonctionnement de l'application, j'ai réalisé une série de tests manuels couvrant tous les cas d'utilisation possibles.

Test effectué	Résultat attendu	Résultat obtenu
Générer avec toutes les options	Mot de passe avec 4 types de caractères	Conforme ✓
Générer sans aucune option cochée	Retour automatique aux minuscules	Conforme ✓
Slider à 4 (minimum)	Mot de passe de 4 caractères	Conforme ✓
Slider à 64 (maximum)	Mot de passe de 64 caractères	Conforme ✓
Cliquer sur Copier	Message 'Copié !' pendant 1,5s	Conforme ✓
Cliquer sur Afficher / Masquer	Bascule visible / caché	Conforme ✓
Générer 11 fois de suite	Historique limité à 10 entrées	Conforme ✓

Test effectué	Résultat attendu	Résultat obtenu
Cliquer sur Effacer l'historique	Liste vide, message de remplacement	Conforme ✓
Vérifier l'indicateur Faible	Court + 1 type → rouge	Conforme ✓
Vérifier l'indicateur Très fort	16+ car + 4 types → vert	Conforme ✓

8.2 Compatibilité navigateurs

J'ai testé l'application sur plusieurs navigateurs pour m'assurer qu'elle fonctionne correctement partout :

Navigateur	Résultat
Google Chrome (v120+)	Fonctionne parfaitement
Mozilla Firefox (v121+)	Fonctionne parfaitement
Microsoft Edge (v120+)	Fonctionne parfaitement
Safari (macOS / iOS)	Fonctionne (API Clipboard OK en HTTPS)
Chrome Mobile (Android)	Interface responsive correcte

Limite identifiée

Sur certains navigateurs mobiles, le bouton Copier peut ne pas fonctionner si la page est ouverte en local (fichier). Ce comportement est normal et lié aux restrictions de sécurité. La solution est de servir la page via un serveur local ou HTTPS.

9. Difficultés rencontrées

Ce projet m'a permis de développer de nombreuses compétences techniques et méthodologiques, grâce aux différents problèmes rencontrés durant sa réalisation. J'ai dû analyser longuement la situation et rechercher des solutions pour améliorer la qualité globale de l'application. Voici les principales difficultés rencontrées :

Difficulté n°1 — L'équilibre de la génération

Au début, il arrivait régulièrement que le mot de passe généré ne contienne aucun chiffre même quand l'option « chiffres » était cochée. Le hasard pur peut en effet produire ce résultat, surtout pour des mots de passe courts. J'ai mis du temps à comprendre pourquoi et à trouver une solution propre.

Difficulté n°2 — Le bouton Copier qui ne fonctionnait pas

En ouvrant le fichier index.html directement depuis mon bureau (protocole file:///), le bouton Copier ne faisait rien de visible. Pas d'erreur, pas de message — c'était très frustrant à déboguer. J'ai finalement découvert que l'API Clipboard nécessite un contexte sécurisé (HTTPS ou localhost) pour fonctionner.

Difficulté n°3 — L'historique qui se dupliquait

À chaque clic sur le bouton Générer, j'ajoutais des éléments HTML à la liste sans jamais effacer les précédents. Résultat : la liste grossissait indéfiniment et contenait des doublons. C'est un bug classique quand on n'a pas l'habitude de gérer le rendu dynamique du DOM.

Difficulté n°4 — Le responsive sur mobile

La grille des quatre options débordait de l'écran sur téléphone. Les textes se chevauchaient et le rendu était vraiment mauvais. Adapter le CSS pour les petits écrans m'a pris plus de temps que prévu.

Difficulté n°5 — L'animation du mot de passe

Je voulais un effet de "scan" au moment où le mot de passe apparaît. Le problème, c'est que si on clique plusieurs fois rapidement, l'animation ne se relançait pas car la classe CSS était déjà présente. J'ai dû apprendre le concept de reflow pour contourner ce problème.

10. Solutions apportées

Pour chaque difficulté rencontrée, j'ai recherché et appliqué une solution adaptée. Voici comment j'ai résolu chaque problème :

Solution n°1 — Fonction d'équilibre

J'ai créé une fonction complémentaire appelée après la génération principale. Elle parcourt la liste des types activés, et pour chacun, force l'insertion d'un caractère de ce type à une position précise dans le mot de passe. Cette approche garantit qu'au moins un caractère de chaque type est toujours présent.

Solution n°2 — Gestion de l'erreur Clipboard

J'ai encadré l'appel à `navigator.clipboard.writeText()` dans un bloc try/catch. En cas d'échec, une erreur est affichée dans la console du navigateur. J'ai aussi ajouté un commentaire dans le code pour expliquer que le projet doit être servi via un serveur local (ex : Live Server de VS Code) ou en HTTPS.

Solution n°3 — Reconstruction propre de l'historique

Avant chaque mise à jour de la liste, je vide complètement son contenu : `historyList.innerHTML = ''`. Ensuite, je reconstruis la liste entièrement depuis le tableau JavaScript. C'est simple, lisible et efficace. La liste est toujours cohérente avec les données.

Solution n°4 — Responsive avec CSS moderne

J'ai utilisé la propriété CSS `min()` pour adapter automatiquement la largeur de la carte. Pour la grille des options, j'ai ajouté un media query qui passe la grille à une seule colonne sur les petits écrans. Le résultat est propre sur tous les formats d'écran testés.

Solution n°5 — Relance de l'animation par reflow

Pour relancer une animation CSS, il faut d'abord retirer la classe, puis forcer le navigateur à recalculer le style. J'ai utilisé l'astuce `void element.offsetWidth;` qui déclenche un reflow. Ensuite, on rajoute la classe et l'animation repart depuis le début.

11. Résultat final

L'application est aujourd'hui complète, fonctionnelle et testée. Elle répond à l'ensemble des fonctionnalités définies dans le cahier des charges initial, y compris les bonus. Elle s'ouvre directement dans le navigateur et fonctionne aussi bien sur ordinateur que sur téléphone.

Récapitulatif des fonctionnalités disponibles

✓ Génération aléatoire d'un mot de passe dès l'ouverture de la page ✓ Curseur de longueur de 4 à 64 caractères avec mise à jour en temps réel ✓ 4 options de caractères : majuscules, minuscules, chiffres, symboles ✓ Garantie d'équilibre : au moins un caractère de chaque type activé ✓ Indicateur de force sur 4 niveaux avec couleurs distinctes et transitions fluides ✓ Bouton Copier avec message de confirmation 'Copié !' pendant 1,5 secondes ✓ Bouton Afficher / Masquer le mot de passe ✓ Historique des 10 derniers mots de passe avec copie individuelle ✓ Interface responsive fonctionnant sur ordinateur et mobile ✓ Code structuré, commenté et entièrement en HTML/CSS/JavaScript

Fonctionnalité	Statut
Génération aléatoire	Réalisée
Choix de la longueur (slider)	Réalisée
Options de caractères (4 types)	Réalisée
Bouton Copier + feedback visuel	Réalisée
Indicateur de force (4 niveaux)	Réalisée
Mise à jour en temps réel	Réalisée
Bouton Afficher / Masquer	Réalisée (bonus)
Historique des 10 derniers	Réalisée (bonus)
Effacer l'historique	Réalisée (bonus)
Interface responsive	Réalisée
Code commenté niveau BTS	Réalisée

12. Conclusion et perspectives

Ce projet a été une vraie expérience d'apprentissage. À travers sa réalisation, j'ai consolidé mes bases en développement web front-end, appris à gérer des problèmes concrets, et pris l'habitude d'écrire du code propre et commenté.

J'ai beaucoup appris sur la manipulation du DOM en JavaScript : sélectionner des éléments, modifier leur contenu, ajouter et retirer des classes CSS, écouter des événements... Ce sont des compétences fondamentales pour tout développeur web.

J'ai aussi trouvé ça intéressant de me confronter à des APIs natives du navigateur comme Clipboard, que je ne connaissais pas avant ce projet. Comprendre pourquoi elle ne fonctionnait pas en local m'a appris quelque chose d'important sur la sécurité des contextes web.

Enfin, ce projet m'a donné envie d'aller plus loin. Un générateur de mots de passe semble simple au premier abord, mais les problèmes qu'on rencontre en le développant sont très concrets et représentatifs de ce qu'on fait dans le métier.

Améliorations envisageables

- LocalStorage : sauvegarder l'historique pour le conserver après un rechargement de page
- Exclusion de caractères ambigus : option pour exclure 0/O, 1/l, I qui se ressemblent
- Génération multiple : générer plusieurs mots de passe en une seule fois
- Mode sombre / clair : bouton de bascule entre dark mode et light mode
- Crypto API : utiliser `window.crypto.getRandomValues()` pour une randomisation encore plus sûre
- Export PDF : permettre d'exporter les mots de passe de l'historique dans un fichier
- Gestionnaire intégré : ajouter un système de catégories pour organiser ses mots de passe

Merci pour votre lecture.

BINTOUL Giovanni — Lycée Melkior-Garré—BTSSIOSLAM 2ème année — 2024 – 2025